

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering
UE24CS341A – Software Engineering | Jackfruit Mini-Project

SYNOPSIS / PROJECT PROPOSAL

1. Project Title and Group Details

Field	Details
Project Title	SnakeArcade – Classic Snake Game with Score Tracking and Difficulty Levels
Group Number	Team 1 (Group G-01)
Project ID	SG-2026-01
Course	UE24CS341A – Software Engineering, V Semester
Development Methodology	Agile (Scrum-based, sprint length 1 week)
Complexity	Medium
Technology Stack	C++17 with SFML (Simple and Fast Multimedia Library)

2. Team Profile and Product Ownership

It is expected that every team member is involved in all project activities. The table below assigns one complete, independently demonstrable functional feature to each member, in addition to supporting (secondary) responsibilities. UI design, database/file interaction and unit testing are treated as secondary responsibilities, not substitutes for functional ownership.

Member	USN / SRN	Owned Functionality (Primary)	Secondary Responsibilities
Rohil Thimmanahalli Girish	PES1UG24CS384 / PES1202402234	Snake Movement, Directional Control and Core Game Loop (rendering, frame timing, input handling)	UI/UX layout and rendering with SFML
Sai Harshith R	PES1UG24CS398 / PES1202400250	Food Generation, Collision Detection and Score Calculation	Unit testing and CI pipeline setup
Sarayu Mayakonda	PES1UG24CS926 / PES1202404807	Game State Management (Start / Pause / Game Over), Difficulty Levels and High-Score Persistence	File/data storage for high-score records, API and code review

3. Problem Statement

Classic arcade-style games such as Snake remain popular as lightweight, low-resource entertainment applications, and are widely used as teaching examples for core software engineering concepts such as real-time input handling, collision detection and state management. However, many simple implementations are monolithic single-file programs with no persistent scoring, no difficulty scaling and no clean separation between game logic and rendering, which makes them hard to extend, test or maintain. SnakeArcade addresses this by building the game as a modular, testable C++ application with SFML-based graphics, configurable difficulty, and a persistent high-score record, while following a proper software engineering lifecycle (requirements, design, implementation, testing and validation).

4. Proposed Project Description

SnakeArcade is a desktop 2D arcade game built using C++ and the SFML graphics library. The player controls a snake that moves continuously on a bounded grid, consumes randomly-spawned food items to grow in length and earn points, and must avoid colliding with the walls or its own body. The system supports multiple difficulty levels that change the

snake's movement speed, tracks the player's current score during a session, and persists the highest score achieved across sessions to a local file so it can be displayed on subsequent runs. The game includes a start screen, an active gameplay state, a pause state and a game-over state with a retry option, giving the player a complete, self-contained arcade experience. This is a new, self-contained project — the team does not have an external customer; the primary stakeholder is the course instructor evaluating the submission, and the intended end users are casual desktop-game players.

5. Target Users / Stakeholders

User Class	Description	Key Interactions
Player	Any casual user running the desktop application to play a single-player game session	Start game, control snake direction, pause/resume, view score, restart after game over
Course Evaluator (Stakeholder)	Faculty member assessing the project against the SE mini-project rubric	Reviews documentation, runs the build, inspects code, test evidence and Git history

6. Objectives

- Deliver a fully playable, crash-free Snake game with smooth, frame-rate-independent movement.
- Provide accurate collision detection for walls, self-collision and food consumption.
- Support at least three selectable difficulty levels that scale snake speed.
- Persist and display the all-time high score across application restarts.
- Maintain a modular code structure (separate modules for input, game logic, rendering and persistence) to support unit testing and future extension.
- Develop the product incrementally using Agile practices with sprint-based delivery and Git-tracked contribution from every member.
- Achieve automated unit test coverage for core game-logic modules (movement, collision, scoring) and demonstrate CI execution on every push.

7. Scope

In Scope	Out of Scope
Single-player Snake gameplay on a fixed-size grid	Multiplayer / networked gameplay
Keyboard-based directional control	Mobile touch or gamepad support
Randomized food spawning and growth mechanic	Procedurally generated levels or obstacles beyond walls
Score tracking during a session	Online leaderboards / cloud sync
Persistent local high-score storage (file-based)	User accounts, login or profile management
Selectable difficulty levels (speed scaling)	In-game purchases or monetization
Start / pause / game-over UI states	Advanced graphics (3D, particle engines, animations beyond sprite movement)

8. Functional Features

At the conclusion of the project, the user will be able to:

1. Start a new game session from a start screen.
2. Control the snake's direction using keyboard input (up / down / left / right).
3. Grow the snake by consuming randomly-spawned food items.

4. View the current score updated live during gameplay.
5. Have the game end automatically on wall collision or self-collision, with a clear game-over screen.
6. Pause and resume an active game session.
7. Select a difficulty level (Easy / Medium / Hard) that changes snake movement speed.
8. View the persisted all-time high score, updated automatically when a new high score is achieved.
9. Restart the game directly from the game-over screen.

9. Plan of Work (Short-Term Plan)

The plan below lists the items the team intends to accomplish over the next six weekly sprints, following the WP1–WP8-style breakdown used in the reference guidelines.

Sprint	Planned Output	Owner(s)	Acceptance Evidence
Sprint 1	Project setup, repo/CI scaffolding, SRS baseline, SFML window + game loop skeleton	All / Rohil	Git repo, user stories, running blank window
Sprint 2	Snake movement, directional control, grid rendering	Rohil	Demo: snake moves and renders correctly + unit tests
Sprint 3	Food spawning, collision detection (wall/self/food), scoring	Sai Harshith	Integration test + PRs + unit tests for collision logic
Sprint 4	Game state management: start, pause, game-over, restart	Sarayu	System demo + test evidence
Sprint 5	Difficulty levels, high-score persistence (file I/O), UI polish	Sarayu / Rohil	Validation + coverage report
Sprint 6	System validation, bug fixes, final report and demo prep	All	Final report + working demo

10. Proposed Technology / Tool Stack

Activity	Tool	Evidence to Submit
Programming language / graphics	C++17 with SFML 2.6	Source code, build artifacts
Build system	CMake	CMakeLists.txt, build logs
Version control	Git + GitHub	Repository URL, commit history
Agile planning	GitHub Projects (Kanban/Scrum board)	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Pipeline run logs / screenshots
Unit testing	GoogleTest (gtest)	Test reports, coverage report
Static analysis	Cppcheck / SonarCloud	Analysis report
Design / UML	Draw.io / PlantUML	UML diagrams

11. Agile Justification

Agile (Scrum) is well suited to this project because the feature set naturally decomposes into short, independently demonstrable increments (movement, collision, scoring, state management, persistence), each of which can be developed, tested and reviewed within a single one-week sprint. Weekly sprint reviews allow early detection of integration issues between the rendering loop and the game-logic modules, and the backlog can be re-prioritized if a feature (e.g., difficulty scaling) proves more complex than estimated, without disrupting the overall delivery timeline.